

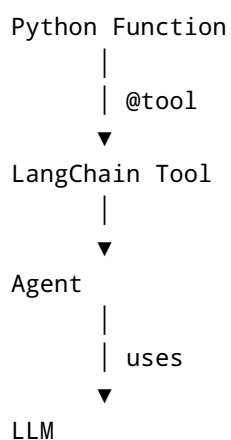
LangChain, LCEL, Tools, and Agent Classes

Python Practice Notes

A practical study guide for understanding how **LCEL**, **LangChain Tools**, and a custom **Agent class** fit together.

1. The Big Picture

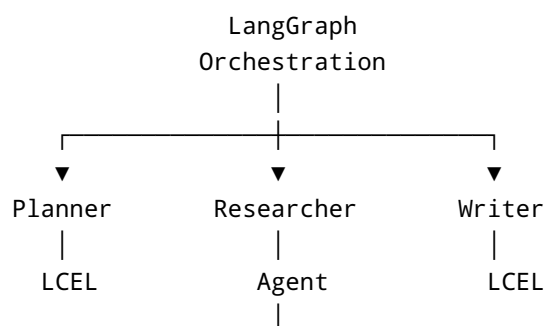
There are four different ideas to keep separate:

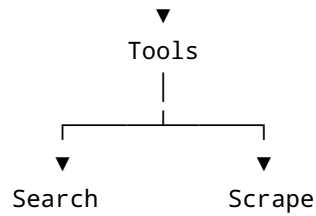


And LCEL is used to connect LangChain components:

Prompt → LLM → Parser

For a larger application:





Think of the responsibilities as:

Component	Job
Python	Basic programming
LangChain	LLM and tool components
LCEL	Connect LangChain components
@tool	Turn Python functions into LangChain tools
Agent class	Your reusable agent abstraction
LangGraph	Control complex workflows

2. What Is LCEL?

LCEL means:

LangChain Expression Language

It lets you connect components using the pipe operator:

```
|
```

The basic pattern is:

```
chain = prompt | llm | parser
```

Read that as:

```
prompt
  ↓
LLM
  ↓
```

```
parser
↓
result
```

3. First LCEL Example

Install the relevant packages:

```
pip install langchain-core langchain-ollama
```

Create a model:

```
from langchain_ollama import ChatOllama

llm = ChatOllama(
    model="qwen2.5-coder:7b",
    temperature=0.1,
)
```

Create a prompt:

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful Python teacher."),
    ("human", "{question}"),
])
```

Create a chain:

```
chain = prompt | llm
```

Run it:

```
result = chain.invoke({
    "question": "Explain Python classes."
})
```

```
print(result.content)
```

The important line is:

```
chain = prompt | llm
```

4. Adding an Output Parser

LLMs often return an AI message object.

You can convert that into a normal string.

```
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()

chain = prompt | llm | parser
```

Now:

```
result = chain.invoke({
    "question": "What is a Python class?"
})

print(result)
```

The flow is:

```
Input
|
▼
ChatPromptTemplate
|
▼
ChatOllama
|
▼
StrOutputParser
|
```

▼
String

5. Why Use LCEL?

Without LCEL, you might manually write:

```
messages = prompt.invoke({
    "question": "What is Python?"
})

response = llm.invoke(messages)

result = parser.invoke(response)
```

With LCEL:

```
chain = prompt | llm | parser

result = chain.invoke({
    "question": "What is Python?"
})
```

LCEL makes the data flow easier to see.

6. Python Functions

Before understanding tools, understand normal Python functions.

```
def add_numbers(a: int, b: int) -> int:
    return a + b
```

Call it:

```
result = add_numbers(5, 3)

print(result)
```

Output:

8

At this point, this is just Python.

The LLM does not automatically know that this function exists.

7. What Is a Tool?

A tool is basically a function that an agent/LLM can be allowed to use.

Example:

```
def get_current_date():  
    return "2026-08-10"
```

A normal Python program can call it:

```
date = get_current_date()
```

But an LLM needs additional information:

```
Tool name  
Tool description  
Arguments  
How to execute it
```

LangChain provides a standard way to package this information.

8. The `@tool` Decorator

LangChain provides:

```
from langchain_core.tools import tool
```

Then:

```
@tool
def get_current_date() -> str:
    """Return the current date."""
    return "2026-08-10"
```

The `@tool` decorator turns the Python function into a LangChain Tool.

Conceptually:

```
Python function
  |
  | @tool
  ▼
LangChain Tool
  |
  ├── name
  ├── description
  ├── arguments
  └── function
```

9. Tool With Arguments

```
from langchain_core.tools import tool

@tool
def calculator(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b
```

The type hints:

```
a: int
b: int
```

help LangChain understand the expected arguments.

The docstring:

```
"""Add two numbers."""
```

helps describe the tool to the model.

10. Calling a Tool Yourself

A LangChain tool can still be called from Python.

```
result = calculator.invoke({  
    "a": 10,  
    "b": 20,  
})  
  
print(result)
```

Output:

```
30
```

This is important:

A tool does not magically execute itself.

Something has to call it.

That something could be:

```
Your Python code  
    OR  
An Agent  
    OR  
LangGraph
```

11. Giving Tools to an LLM

Create some tools:

```

from langchain_core.tools import tool

@tool
def get_current_date() -> str:
    """Return today's date."""
    return "2026-08-10"

@tool
def calculator(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

```

Create the LLM:

```

from langchain_ollama import ChatOllama

llm = ChatOllama(
    model="qwen2.5-coder:7b",
    temperature=0.1,
)

```

Bind the tools:

```

llm_with_tools = llm.bind_tools([
    get_current_date,
    calculator,
])

```

Now the model knows about the available tools.

12. Tool Calling

You can ask:

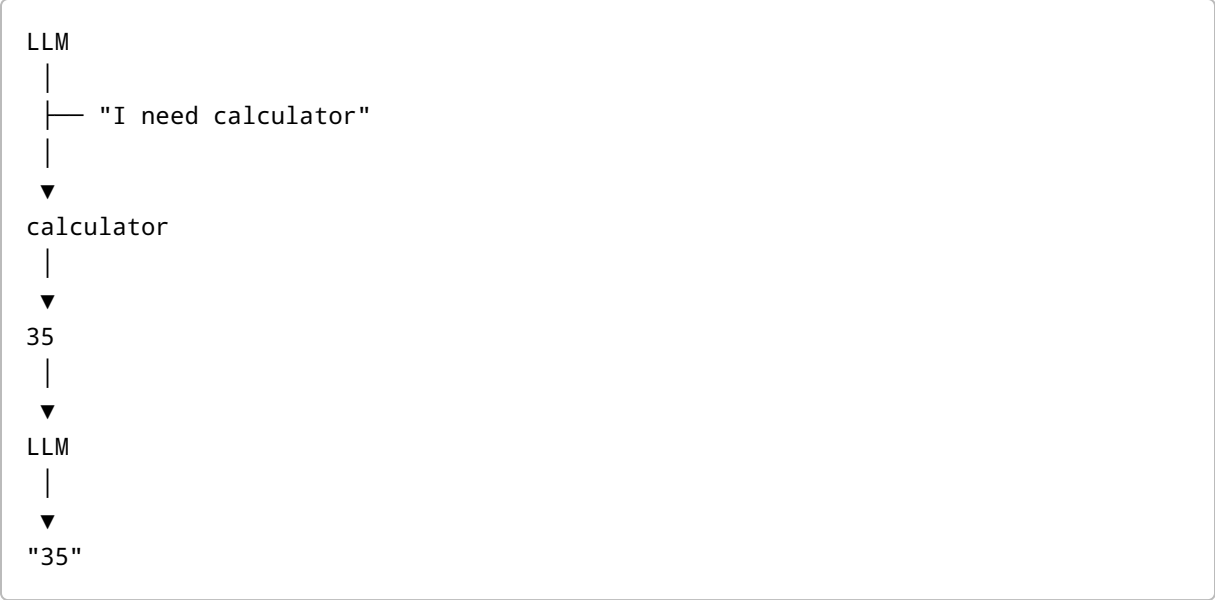
```

response = llm_with_tools.invoke(
    "What is 10 plus 25?"
)

```

The model may return a tool call rather than immediately returning:

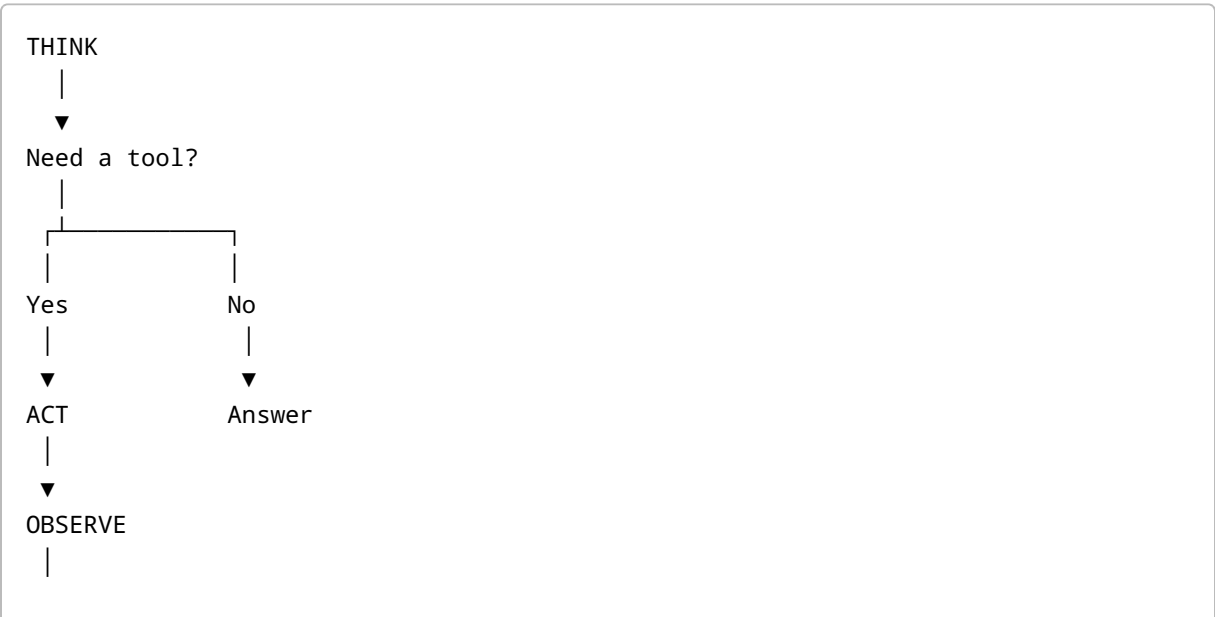
Conceptually:



This is the fundamental agent/tool loop.

13. The Agent Loop

An agent generally does something like:





THINK

This is why your `Agent` class has:

```
think()  
act()  
observe()
```

Those methods map nicely onto the basic agent cycle.

14. Your AgentProfile

Your profile describes the identity/instructions of the agent.

```
from dataclasses import dataclass  
  
@dataclass  
class AgentProfile:  
    name: str  
    system_prompt: str
```

Create profiles:

```
planner_profile = AgentProfile(  
    name="Planner",  
    system_prompt="""  
    You are a research planning agent.  
    Create clear, practical research plans.  
    """  
)
```

Another:

```
researcher_profile = AgentProfile(  
    name="Researcher",  
    system_prompt="""  
    You are a research agent.  
    Use tools to gather useful information.
```

```
)
```

15. Your AgentState

State contains the information accumulated while the agent runs.

```
from dataclasses import dataclass, field

@dataclass
class AgentState:
    messages: list = field(default_factory=list)
    tool_results: list[str] = field(default_factory=list)
```

Think of it as the agent's temporary workspace.

```
AgentState
|
├── messages
|
└── tool_results
```

16. Your Agent Class

A simplified version:

```
class Agent:

    def __init__(self, llm, tools, profile):
        self.llm = llm
        self.profile = profile
        self.tools = {
            tool.name: tool
            for tool in tools
        }

    def start(self, user_input: str):
        return AgentState([
```

```

        {
            "role": "system",
            "content": self.profile.system_prompt,
        },
        {
            "role": "user",
            "content": user_input,
        },
    ])

    def think(self, state):
        model = self.llm.bind_tools(
            list(self.tools.values())
        )

        response = model.invoke(
            state.messages
        )

        state.messages.append(response)

        return response

```

The important relationship is:

```

Agent
├── profile
├── llm
├── tools
└── state

```

17. Creating an Agent

```

researcher = Agent(
    llm=llm,
    tools=[
        get_current_date,
        calculator,
    ]
)

```

```
],  
    profile=researcher_profile,  
)
```

Now you have a reusable researcher.

You can create another one:

```
writer = Agent(  
    llm=llm,  
    tools=[],  
    profile=writer_profile,  
)
```

Same class.

Different configuration.

18. Why This Is Useful

Instead of creating:

```
class PlannerAgent:  
    ...  
  
class ResearcherAgent:  
    ...  
  
class WriterAgent:  
    ...
```

you can have:

```
class Agent:  
    ...
```

and configure it:

```

planner = Agent(
    llm=llm,
    tools=[],
    profile=planner_profile,
)

researcher = Agent(
    llm=llm,
    tools=[web_search, scrape_url],
    profile=researcher_profile,
)

writer = Agent(
    llm=llm,
    tools=[],
    profile=writer_profile,
)

```

The class provides the behavior.

The profile provides the identity.

The tools provide the capabilities.

19. Agent + @tool

This is the combination you were asking about.

Create tools:

```

from langchain_core.tools import tool

@tool
def web_search(query: str) -> str:
    """Search the web for information."""
    return f"Results for: {query}"

@tool
def scrape_url(url: str) -> str:

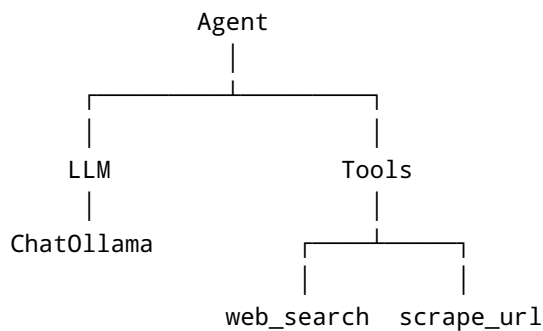
```

```
"""Retrieve text from a webpage."""  
return f"Contents of: {url}"
```

Create the agent:

```
researcher = Agent(  
    llm=llm,  
    tools=[  
        web_search,  
        scrape_url,  
    ],  
    profile=researcher_profile,  
)
```

The relationship is:



20. Agent + LCEL

LCEL and your Agent do not have to compete.

You can use LCEL for simple deterministic chains:

```
planner = prompt | llm | parser
```

And use your Agent when tools are required:

```
researcher = Agent(  
    llm=llm,  
    tools=[web_search, scrape_url],
```

```
    profile=researcher_profile,  
)
```

This gives you two different patterns:

Simple task:

Prompt → LLM → Parser

Tool-using task:

Agent

↓

LLM

↓

Tool

↓

LLM

↓

Answer

21. LCEL Is Not the Same as an Agent

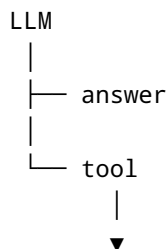
This distinction is important.

LCEL:

```
chain = prompt | llm | parser
```

is primarily a **data pipeline**.

An agent:



```
result
|
▼
LLM
```

is a **decision-making loop**.

LCEL says:

"Pass this output into the next component."

An agent says:

"Decide what to do next."

22. LCEL Practice Exercise

Create this:

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_ollama import ChatOllama

llm = ChatOllama(
    model="qwen2.5-coder:7b",
    temperature=0.1,
)

prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        "You are a Python teacher."
    ),
    (
        "human",
        "Explain {topic} in simple terms."
    ),
])

parser = StrOutputParser()

chain = prompt | llm | parser
```

```
result = chain.invoke({
    "topic": "Python classes"
})

print(result)
```

Practice

Change:

```
"Python classes"
```

to:

```
decorators
functions
dataclasses
LCEL
LangChain tools
agents
LangGraph
```

23. Tool Practice Exercise

Create:

```
@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b
```

Test it:

```
result = multiply.invoke({
    "a": 5,
    "b": 10,
})
```

```
print(result)
```

Expected:

```
50
```

Then create:

```
@tool
def reverse_text(text: str) -> str:
    """Reverse the supplied text."""
    return text[::-1]
```

Test:

```
print(
    reverse_text.invoke({
        "text": "hello"
    })
)
```

Expected:

```
olleh
```

24. Agent Practice Exercise

Create a profile:

```
researcher_profile = AgentProfile(
    name="Researcher",
    system_prompt="""
    You are a research assistant.
    Use available tools when they are useful.
    """
)
```

Create an agent:

```
researcher = Agent(  
    llm=llm,  
    tools=[  
        web_search,  
        scrape_url,  
    ],  
    profile=researcher_profile,  
)
```

Start it:

```
state = researcher.start(  
    "Research the latest AI agent architectures."  
)
```

Then think:

```
response = researcher.think(state)  
  
print(response)
```

Inspect:

```
print(state.messages)
```

25. Understand the Three Layers

When practicing, keep these three things separate.

Layer 1 — Function

```
def calculate(a, b):  
    return a + b
```

Normal Python.

Layer 2 — Tool

```
@tool
def calculate(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b
```

LangChain understands the function as a tool.

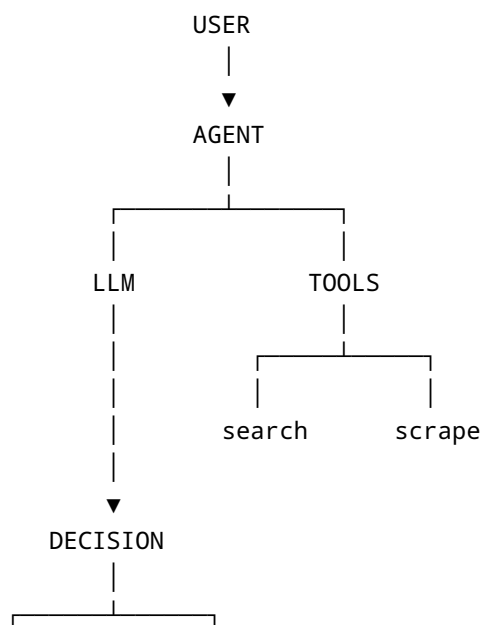
Layer 3 — Agent

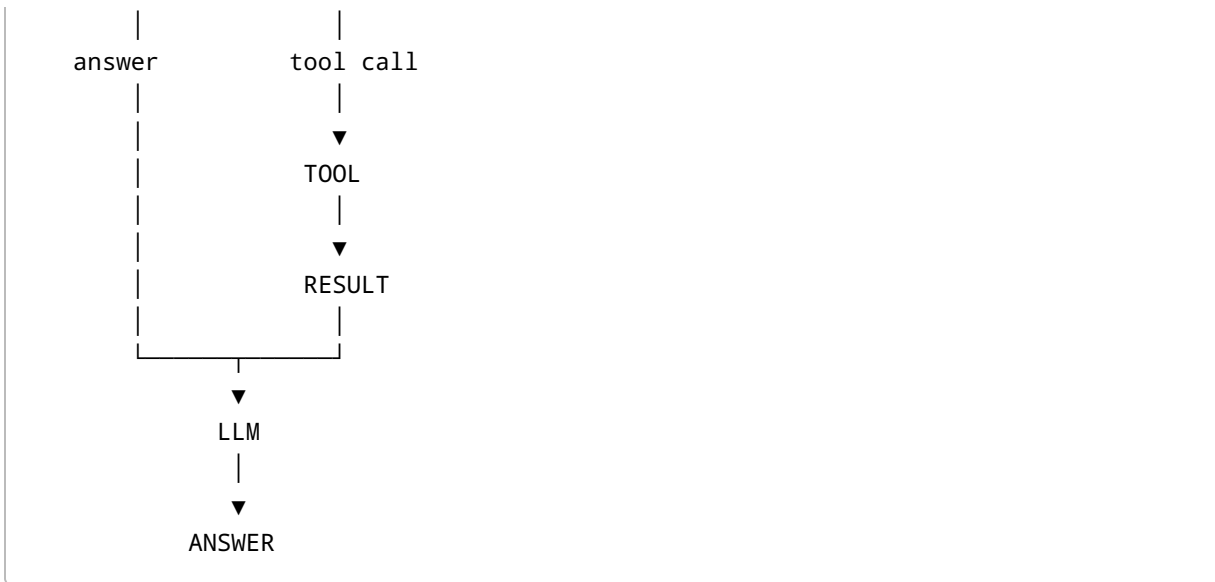
```
agent = Agent(
    llm=llm,
    tools=[calculate],
    profile=profile,
)
```

The agent can give the LLM access to the tool.

26. The Complete Mental Model

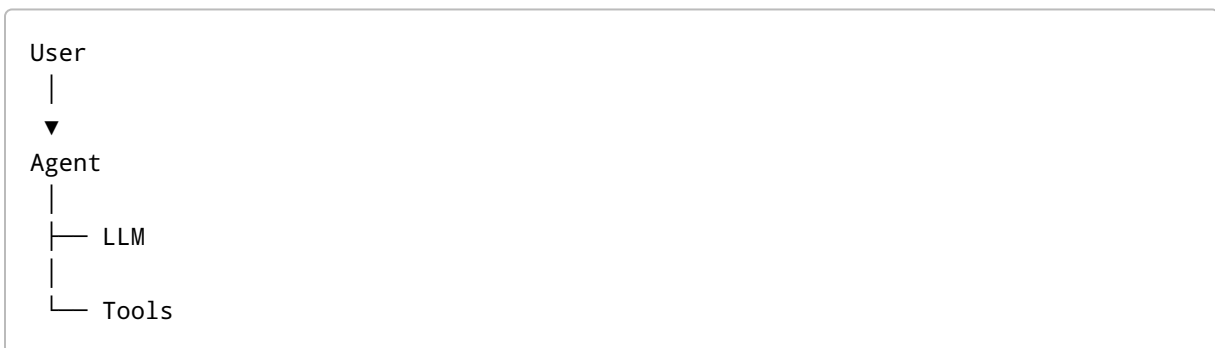
Memorize this:





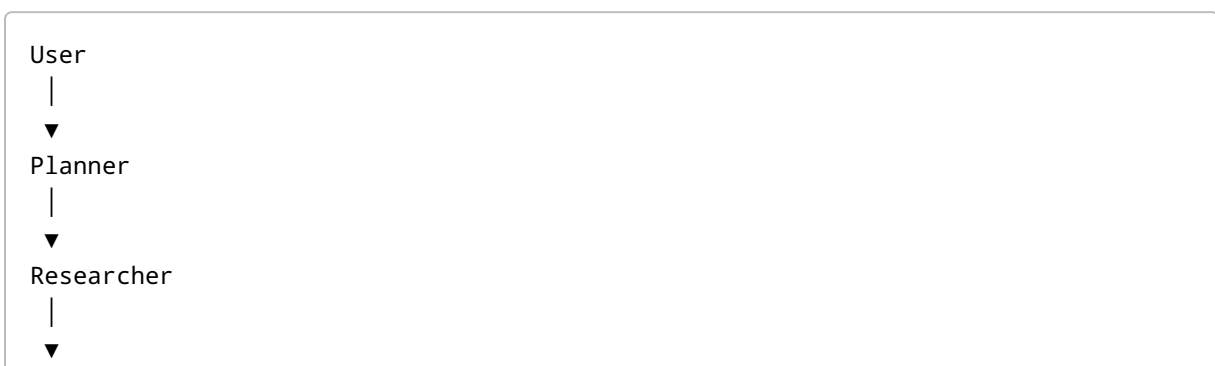
27. Where LangGraph Fits

When you have one agent:



You may not need LangGraph yet.

When you have multiple agents:



```
Reviewer
|
▼
Writer
```

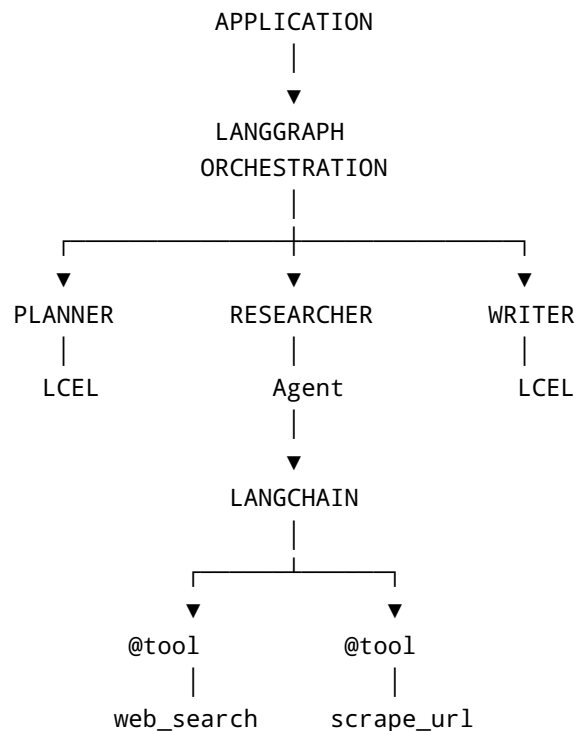
LangGraph becomes useful.

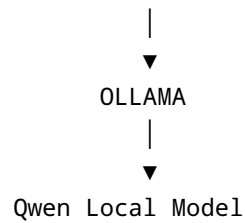
It controls:

```
state
routing
loops
conditions
agent-to-agent communication
tool execution
checkpoints
human approval
```

28. Your Target Architecture

For your research application, a good architecture is:





This gives each technology a clear responsibility.

29. Quick Reference

LCEL

```
chain = prompt | llm | parser
```

Think:

```
connect components
```

@tool

```
@tool
def search(query: str):
    """Search the web."""
    ...
```

Think:

```
turn Python function into a LangChain tool
```

Agent

```
agent = Agent(
    llm=llm,
    tools=[search],
    profile=profile,
)
```

Think:

```
LLM + identity + capabilities
```

Agent State

```
state = AgentState()
```

Think:

```
what the agent currently knows
```

LangGraph

```
Planner → Researcher → Writer
```

Think:

```
control how work moves
```

Ollama

```
ChatOllama → local Qwen model
```

Think:

```
where the LLM runs
```

30. Practice Order

Don't try to learn everything at once.

Practice in this order:

```
1. Python functions
```

```
↓
```

```
2. Python classes
   ↓
3. @ decorators
   ↓
4. LangChain @tool
   ↓
5. ChatOllama
   ↓
6. LCEL
   ↓
7. Tool calling
   ↓
8. Your Agent class
   ↓
9. Agent + tools
   ↓
10. LangGraph
    ↓
11. Multiple agents
```

The most important progression is:

```
Python Function
   ↓
@tool
   ↓
LangChain Tool
   ↓
Agent
   ↓
Tool-using Agent
   ↓
LangGraph
   ↓
Multi-Agent System
```

That progression will make the architecture much easier to understand because each layer adds **one new responsibility** rather than introducing LangChain, LCEL, tools, agents, and LangGraph all at the same time.